PARCOURS GUIDÉ • DÉVELOPPEMENT PYTHON • DATA VISUALISATION

# ATLAS COMMERCE

## CONSTRUIRE SON PREMIER DASHBOARD DÉCISIONNEL AVEC PLOTLY DASH

Un TP réellement pas à pas, du dossier vide à l’application interactive.

**01**
Créer son environnement

**02**
Découvrir Dash

**03**
Construire Atlas Commerce

**10 heures · Niveau intermédiaire en Python**

Guide stagiaire + 12 checkpoints de code + dataset métier et contrôles

Version revue : 23 octobre 2026

# Comment travailler avec ce guide

Ce document est volontairement **très guidé**. Il ne suffit pas de lire les scripts : il faut les écrire, les exécuter et interpréter le résultat à chaque petite étape. L'apprenant ne commence jamais par une application de 300 lignes. Il la construit par ajouts successifs, sous des repères explicites.

| Temps | Phases | Production vérifiable |
|---|---|---|
| 1 h | Installation VS Code, Python, venv, bibliothèques | `test.py` exécuté par le bon interpréteur |
| 3 h | Dash pas à pas : page, slider, callback, graphiques, dropdown | `decouverte.py` réactif |
| 6 h | Atlas Commerce : audit, KPI, cinq graphiques et filtrage croisé | `app.py` complet et contrôlé |

## La méthode de travail : toujours cinq actions

1. **LOCALISER :** ouvrez le fichier demandé, cherchez le commentaire `# ZONE_...`.
2. **ÉCRIRE :** copiez le bloc indiqué *sous* le commentaire. Si le bloc est divisé en plusieurs parties, collez-les dans le même ordre.
3. **ENREGISTRER :** utilisez `Ctrl + S`.
4. **EXÉCUTER :** dans le terminal, lancez la commande demandée. Le navigateur se rafraîchit souvent tout seul en mode debug.
5. **VALIDER :** comparez votre écran avec la rubrique « Résultat attendu » et répondez à une question métier.

## Le système de secours : 12 checkpoints exécutables

Dans le fichier ZIP associé, le dossier `checkpoints` contient `00_depart.py` jusqu'à `11_analyse_metier.py`. Chaque checkpoint contient le **code complet à ce stade**. Si vous êtes bloqué, comparez votre fichier avec ce checkpoint ; en dernier recours, copiez le checkpoint à la racine du projet, puis renommez-le `app.py`. Les scripts des checkpoints ne sont pas à lancer directement dans leur sous-dossier : la copie à la racine facilite l'import de `donnees.py`.

> **Règle essentielle**
>
> Ne lancez jamais à la fois `decouverte.py` et `app.py` sur le port 8050 : arrêtez le premier serveur avec `Ctrl + C`. Pour éviter les erreurs, vous ne gardez qu'un serveur actif.

## Carte du parcours Atlas Commerce

| Checkpoint | Action concrète | Ce qui apparaît |
|---|---|---|
| 00 | Ouvrir le squelette | Page vide mais serveur fonctionnel |
| 01 | Insérer l'en-tête | Bandeau bleu Atlas Commerce |
| 02 | Insérer les filtres | Dates, catégorie, Jour/Mois |
| 03 | Créer quatre cartes | Quatre KPI à « -- » |
| 04 | Écrire le callback KPI | Quatre valeurs qui réagissent |
| 05 | Ajouter la courbe du CA | Évolution jour/mois |
| 06 | Ajouter les catégories | Barres de CA par catégorie |
| 07 | Ajouter les régions | Donut régional |

| | | |
|---|---|---|
| 08 | Ajouter les marges | Barres de marge commerciale |
| 09 | Ajouter les produits | Top 10 horizontal |
| 10 | Créer le filtrage croisé | Clic régional + bouton de remise à zéro |
| 11 | Ajouter le commentaire métier | Synthèse lisible pour un décideur |

## Ce que vous devez savoir avant de commencer

Prérequis : utiliser des variables, fonctions, conditions, listes et dictionnaires en Python ; avoir déjà manipulé un DataFrame pandas. Aucun prérequis en HTML, CSS ni JavaScript n’est exigé. Le projet sera développé dans **Visual Studio Code**, sous Windows (commandes PowerShell indiquées) ou macOS/Linux (variantes indiquées).

Données et limites : il s’agit d’une entreprise fictive, avec un CSV synthétique et une marge commerciale simplifiée ; nous n’interprétons pas les chiffres comme des données en temps réel ou comme une marge nette.

PHASE 1 · 60 MINUTES

# Préparer l’environnement sans se perdre

Résultat : le bon interpréteur Python, un dossier de projet et des bibliothèques installées dans un environnement isolé.

MISSION P1-01 · 10 MIN · FICHIER : TERMINAL VS CODE

# Vérifier Python et découvrir le terminal

**Votre objectif**

Faire exécuter une commande simple par VS Code.

**Livrable visible :** La version Python apparaît.

1. Ouvrez **Visual Studio Code**.
2. En haut, cliquez sur **Terminal → New Terminal**.
3. Dans la zone qui apparaît en bas, cliquez après le symbole `>` ou `$`.
4. Saisissez exactement la commande suivante, puis appuyez sur Entrée.

```
python --version
```

Sur Windows, si la commande échoue, essayez `py --version`. Sous macOS/Linux, essayez `python3 --version`. Le prérequis conseillé pour ce TP est Python 3.11 ou 3.12.

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Exemple : `Python 3.12.x`. Le numéro mineur peut être différent. Le terminal ne doit pas afficher « commande introuvable ».

**QUESTION / MINI-DÉFI**

Dans votre cahier, différenciez Python (langage/interpréteur) et VS Code (éditeur/IDE).

MISSION P1-02 · 10 MIN · FICHIER : VS CODE

# Créer le dossier et installer l’extension Python

**Votre objectif**

Savoir ouvrir un projet et rendre VS Code capable de l’exécuter.

**Livrable visible :** Le dossier `dashboard-commerce` apparaît à gauche.

1. Sur votre ordinateur, créez un dossier nommé `dashboard-commerce`.
2. Dans VS Code, cliquez sur **File → Open Folder** et sélectionnez ce dossier.
3. Ouvrez la barre **Extensions** (`Ctrl + Shift + X`).
4. Cherchez `Python`, choisissez l’extension publiée par **Microsoft**, puis cliquez sur **Install**.
5. Dans l’explorateur de VS Code, vérifiez que le dossier ouvert est bien `dashboard-commerce`.

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l’obtenir**

L’explorateur latéral montre `dashboard-commerce` ; aucun autre projet n’est ouvert par erreur.

MISSION P1-03 · 15 MIN · FICHIER : VS CODE → PALETTE

# Créer et sélectionner l’environnement virtuel

> **Votre objectif**
>
> Isoler les bibliothèques du TP.
>
> **Livrable visible :** Un dossier `.venv` est créé et sélectionné.

1. Dans VS Code, appuyez sur `Ctrl + Shift + P`.
2. Écrivez `Python: Create Environment`.
3. Choisissez **Venv**, puis votre Python 3.11/3.12.
4. Attendez la création du dossier `.venv`.
5. Revenez à `Ctrl + Shift + P` → `Python: Select Interpreter`.
6. Sélectionnez le chemin qui contient `dashboard-commerce/.venv`.
7. Ouvrez un **nouveau terminal** pour vérifier son activation automatique.

Si le terminal n’affiche pas `(.venv)`, activez-le manuellement :

```
Windows PowerShell :  .\.venv\Scripts\Activate.ps1
Windows CMD :         .venv\Scripts\activate
macOS/Linux :         source .venv/bin/activate
```

> **RÉSULTAT ATTENDU - ne passez pas à la suite avant de l’obtenir**
>
> Exemple : `(.venv) PS C:\...\dashboard-commerce>`.

> **SI CELA NE MARCHE PAS**
>
> PowerShell peut interdire les scripts. Ouvrez CMD, ou lancez dans le terminal PowerShell `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` puis réactivez. Cette modification ne concerne que le processus courant.

MISSION P1-04 · 15 MIN · FICHIER : TERMINAL VS CODE

# Installer et contrôler les packages

**Votre objectif**

Disposer des packages exacts de Dash et pandas.

**Livrable visible :** La commande de vérification affiche la version de Dash.

Avant toute commande, vérifiez que vous voyez `(.venv)`. Puis saisissez :

```
python -m pip install --upgrade pip
python -m pip install "dash==4.4.1" pandas plotly dash-bootstrap-components dash-extensions
```

**À quoi servent les packages ?** `dash` : application web et callbacks ; `plotly` : graphiques ; `pandas` : traitement des ventes ; `dash-bootstrap-components` : grille et cartes responsive ; `dash-extensions` : optimisations avancées (utilisée seulement en prolongement).

```
python -m pip show dash
python -c "import dash; print(dash.__version__)"
```

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

La version est `4.4.1`, sans traceback. Si vous travaillez plus tard avec une version différente, reportez-vous à ses notes de version.

**SI CELA NE MARCHE PAS**

Une erreur `ModuleNotFoundError` signifie généralement que le package n'est pas installé dans l'interpréteur actif. Ré-exécutez `python -m pip install ...` après activation de `.venv`.

MISSION P1-05 · 10 MIN · FICHIER : TEST.PY

# Créer le premier fichier Python et le lancer

**Votre objectif**

Vérifier que Python lancé depuis le terminal est celui du projet.

**Livrable visible :** La chaîne Hello Dash et le chemin de `.venv` apparaissent.

1. Dans l'explorateur VS Code, cliquez sur **New File**.
2. Nommez le fichier `test.py` et appuyez sur Entrée.
3. Écrivez le code ci-dessous puis enregistrez avec `Ctrl + S`.

```
import sys
print("Hello Dash")
print("Interpreteur :", sys.executable)
```

```
python test.py
```

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Vous voyez `Hello Dash`, puis un chemin contenant `.venv`.

**QUESTION / MINI-DÉFI**

Pourquoi garder un `.venv` par projet ? Parlez d'isolation, de conflit de versions et de reproductibilité.

PHASE 2 · 180 MINUTES

# Comprendre Dash sur une petite application

Le commerce attendra : nous apprenons d’abord à faire fonctionner une page, un curseur, un callback et deux graphiques.

MISSION P2-01 · 30 MIN · FICHIER : DECOUVERTE.PY

# Premier serveur et première page

**Votre objectif**

Comprendre app, layout et serveur local.

**Livrable visible :** Une page avec titre et paragraphe à l'adresse locale.

1. Dans `dashboard-commerce`, créez `decouverte.py`.
2. Copiez tout le bloc suivant.
3. Enregistrez, puis exécutez `python decouverte.py`.
4. Dans votre navigateur, entrez `http://127.0.0.1:8050`.

```
from dash import Dash, html, dcc, Input, Output
import plotly.express as px

app = Dash(__name__)
app.title = "Decouverte Dash"

app.layout = html.Div([
    html.H1("Mon premier site Dash"),
    html.P("Je construis une page web avec Python.")
])

if __name__ == "__main__":
    app.run(debug=True)
```

`Dash(__name__)` initialise l'application. `app.layout` contient l'arbre des composants. `html.H1` est un titre, `html.P` un paragraphe. `children` est le contenu du composant ; c'est aussi le premier argument positionnel.

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Le navigateur affiche exactement les deux textes. Le terminal indique une adresse locale et le serveur reste en cours d'exécution.

**SI CELA NE MARCHE PAS**

Si la page reste blanche, vérifiez dans le terminal si Python affiche un traceback. Si le port 8050 est occupé, arrêtez un autre serveur avec `Ctrl + C`.

**QUESTION / MINI-DÉFI**

Remplacez le texte de `html.H1` par « Dashboard du futur analyste » puis vérifiez le rechargement automatique.

MISSION P2-02 · 40 MIN · FICHIER : DECOUVERTE.PY

# Créer une interaction avec Slider et callback

**Votre objectif**

Relier un contrôle à une fonction Python puis à une zone HTML.

**Livrable visible :** Le carré de la valeur change sans actualisation manuelle.

1. Dans `decouverte.py`, **repérez** la ligne `app.layout = html.Div([`.
2. **Remplacez tout le layout** (jusqu'à sa parenthèse fermante) par le bloc ci-dessous.
3. Laissez les importations et le bloc `if __name__ == "__main__"` inchangés.

```
app.layout = html.Div([
    html.H1("Mon premier site Dash"),
    html.H2("Le carre d'un nombre"),
    dcc.Slider(
        id="curseur", min=0, max=10,
        step=1, value=4,
        marks={i: str(i) for i in range(0, 11, 2)}
    ),
    html.Div(id="resultat", style={"marginTop": "20px"})
])
```

D'abord, **prédisez** : le curseur va-t-il afficher automatiquement un résultat ? Non : nous avons créé les composants mais pas leur relation.

1. Entre la fin du layout et le `if __name__...`, collez le callback suivant.
2. Enregistrez, puis rechargez la page.

```
@app.callback(
    Output("resultat", "children"),
    Input("curseur", "value")
)
def afficher_carre(valeur):
    return f"Carre de {valeur} = {valeur**2}"
```

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

À la position 4 : `Carre de 4 = 16`. À la position 7 : `Carre de 7 = 49`. Le résultat change immédiatement.

`Input("curseur", "value")` surveille une valeur. `Output("resultat", "children")` indique où écrire. Le `return` fournit exactement une sortie.

**QUESTION / MINI-DÉFI**

Modifiez le callback pour afficher aussi le cube. Indice : il est possible de retourner un `html.Div([html.P(...), html.P(...)])` dans l'unique Output.

MISSION P2-03 · 35 MIN · FICHIER : DECOUVERTE.PY

# Visualiser des données avec Plotly Express

**Votre objectif**

Distinguer la construction de la figure et son affichage avec dcc.Graph.

**Livrable visible :** Un nuage de points Gapminder s'affiche.

1. Arrêtez le serveur avec `Ctrl + C` pour modifier plus facilement le fichier si nécessaire.
2. Sous les importations, ajoutez le chargement de données ci-dessous.
3. Remplacez le layout du slider par le nouveau layout graphique.

**Sous les importations**

```
donnees = px.data.gapminder()
donnees = donnees[donnees["year"] == 2007].copy()
print(donnees[["country", "continent", "lifeExp"]].head())
```

**Remplacement du layout**

```
app.layout = html.Div([
    html.H1("Richesse et esperance de vie en 2007"),
    dcc.Graph(
        id="nuage",
        figure=px.scatter(
            donnees, x="gdpPercap", y="lifeExp",
            size="pop", color="continent",
            hover_name="country", log_x=True, size_max=50
        )
    )
])
```

1. Supprimez le callback du slider (ses identifiants n'existent plus dans le layout).
2. Enregistrez puis relancez `python decouverte.py`.
3. Passez la souris sur une bulle et notez le pays affiché.

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Un graphique avec plusieurs continents apparaît. Les bulles ont différentes tailles et le survol montre les données associées.

**QUESTION / MINI-DÉFI**

À votre avis, pourquoi l'axe PIB par habitant utilise-t-il une échelle logarithmique ?

MISSION P2-04 · 35 MIN · FICHIER : DECOUVERTE.PY

# Filtrer un nuage avec un Dropdown

**Votre objectif**

Réutiliser un Input pour recalculer la propriété figure.

**Livrable visible :** Le continent sélectionné contrôle le graphique.

1. Dans la liste du layout, **avant** `dcc.Graph(id="nuage", ...)`, ajoutez le `dcc.Dropdown` ci-dessous.
2. Pour que ce soit plus lisible, remplacez le `dcc.Graph(..., figure=...)` par `dcc.Graph(id="nuage")` : le callback générera la figure.

**Les composants à placer dans la liste du layout**

```
html.P("Choisissez un continent"),
dcc.Dropdown(
    id="continent",
    options=["Tous"] + sorted(donnees["continent"].unique()),
    value="Tous", clearable=False
),
dcc.Graph(id="nuage")
```

1. Après la fermeture du layout et avant le `if __name__`, ajoutez le callback suivant.

```
@app.callback(Output("nuage", "figure"), Input("continent", "value"))
def filtrer_nuage(continent):
    if continent == "Tous":
        dff = donnees.copy()
    else:
        dff = donnees[donnees["continent"] == continent].copy()
    return px.scatter(
        dff, x="gdpPercap", y="lifeExp", size="pop",
        color="continent", hover_name="country",
        log_x=True, size_max=50
    )
```

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Sélectionnez « Africa » puis « Europe » : les points visibles changent. Revenez sur « Tous » : plusieurs couleurs sont à nouveau affichées.

**SI CELA NE MARCHE PAS**

Vérifiez que `id="continent"` dans le Dropdown et `Input("continent", "value")` dans le callback sont rigoureusement identiques.

MISSION P2-05 · 40 MIN · FICHIER : DECOUVERTE.PY

# Ajouter un second graphique synchronisé

**Votre objectif**

Comprendre que plusieurs composants peuvent réagir au même Input.

**Livrable visible :** Un nuage et un Top 10 réagissent ensemble.

1. Dans le layout, **juste après** `dcc.Graph(id="nuage")`, ajoutez le sous-titre et le second graphique.
2. Ajoutez ensuite le second callback après le premier ; ne supprimez pas le callback du nuage.

**Dans le layout**

```
html.H2("Population : 10 premiers pays"),
dcc.Graph(id="barres")
```

```
@app.callback(Output("barres", "figure"), Input("continent", "value"))
def filtrer_barres(continent):
    if continent == "Tous":
        dff = donnees.copy()
    else:
        dff = donnees[donnees["continent"] == continent].copy()
    top = dff.nlargest(10, "pop").sort_values("pop")
    return px.bar(
        top, x="pop", y="country", orientation="h",
        title="Les pays les plus peuples"
    )
```

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

L'utilisateur voit deux graphiques. Un seul changement de continent déclenche la mise à jour des deux figures.

**QUESTION / MINI-DÉFI**

Mini-défi en binôme : expliquez à voix haute, sans regarder le code, le trajet d'une sélection du Dropdown jusqu'au second graphique. Puis changez les titres des deux figures.

Vous avez appris le mécanisme complet qui sera réutilisé dans Atlas Commerce. Sauvegardez ce travail sous `decouverte.py`, arrêtez le serveur et passez au vrai projet.

PHASE 3 · 360 MINUTES

# Projet fil rouge Atlas Commerce

Une entreprise fictive vend douze produits dans six régions. Vous êtes chargé de livrer un tableau de bord que l'équipe commerciale pourra utiliser.

## Brief du client et découpage des 6 heures

**Message du client :** « Nous recevons beaucoup de lignes de ventes mais nous ignorons quelles régions, catégories et références gagnent réellement de l'argent. Je veux des chiffres fiables, des graphiques qui réagissent aux filtres et un Top produits que je puisse explorer par région. »

| Créneau | Micro-missions | Durée |
|---|---|---|
| 3.1 | CSV, contrôle qualité, nettoyage, fonctions métier | 60 min |
| 3.2 | Squelette, en-tête, filtres, responsive, cartes | 65 min |
| 3.3 | Callback des cartes KPI et vérification | 45 min |
| 3.4 | Courbe, catégories, régions, marges, Top 10 | 100 min |
| 3.5 | Filtrage croisé, recommandation, performances | 60 min |
| 3.6 | Tests, débogage, requirements, soutenance | 30 min |

### Fichiers présents dans les ressources

```
dashboard-commerce/
|-- app_depart.py       # structure a completer
|-- app.py              # solution complete (ne pas regarder d abord)
|-- donnees.py          # chargement / nettoyage / filtres / KPI
|-- generer_donnees.py  # regenerer le CSV
|-- sales_data_brut.csv # donnees fictives avec anomalies
|-- assets/style.css    # apparence professionnelle
|-- checkpoints/        # solutions intermediaires
|-- tests_metier.py     # controles automatises
```

> **Première action dans VS Code**
>
> Si vous utilisez l’archive ZIP, extrayez-la puis ouvrez dans VS Code le dossier `Atlas_Commerce_TP` ou renommez ce dossier `dashboard-commerce`. Tout au long de cette phase, `app.py`, `donnees.py` et `sales_data_brut.csv` doivent être **dans le même dossier racine**.

MISSION 3.1-A · 15 MIN · FICHIER : SALES_DATA_BRUT.CSV

# Observer les ventes avant de coder

**Votre objectif**

Comprendre ce qu'est une ligne de vente et distinguer commande, article, CA et marge.

**Livrable visible :** Les colonnes sont identifiées et trois anomalies recherchées.

1. Dans VS Code, cliquez sur `sales_data_brut.csv`.
2. Regardez l'en-tête et repérez `Ligne_ID`, `Commande_ID`, `Date`, `Categorie`, `Region`, `Quantite`, `Remise_Pct`, `Chiffre_Affaires`.
3. Identifiez deux lignes qui portent le même `Commande_ID` : cela signifie qu'une commande contient plusieurs lignes d'articles.
4. Créez un fichier temporaire `inspection.py` et copiez les instructions ci-dessous.
5. Exécutez `python inspection.py`.

```
import pandas as pd

df = pd.read_csv("sales_data_brut.csv", encoding="utf-8-sig")
print("Dimensions :", df.shape)
print("Colonnes :", df.columns.tolist())
print("Valeurs nulles :")
print(df.isna().sum())
print("IDs dupliques :", df["Ligne_ID"].duplicated().sum())
print("Commandes :", df["Commande_ID"].nunique())
```

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Attendu : 4 784 lignes brutes, 2 400 commandes distinctes, une valeur région manquante et un identifiant de ligne dupliqué. La date invalide n'est pas encore identifiable avec `isna()` tant qu'elle reste une chaîne.

**QUESTION / MINI-DÉFI**

Pourquoi serait-il faux d'utiliser `len(df)` comme nombre de commandes ? Et pourquoi ne suffit-il pas de supprimer les lignes avec une région vide ?

MISSION 3.1-B · 30 MIN · FICHIER : DONNEES.PY

# Construire donnees.py morceau par morceau

> **Votre objectif**
>
> Écrire le nettoyage et les fonctions métier, sans sauter directement au script final.
>
> **Livrable visible :** Un fichier pandas de 83 lignes, construit en cinq insertions.

Cette mission est volontairement plus détaillée : **vous allez réellement écrire le fichier de préparation**. Si l'archive fournit déjà `donnees.py`, renommez-le d'abord `donnees_solution.py`, puis créez un nouveau fichier vide nommé `donnees.py`. Ne mélangez pas les lignes des deux fichiers.

## Insertion 1/5 - importer les données et contrôler les noms des colonnes

1. Dans l'explorateur VS Code, cliquez sur **New File**, saisissez `donnees.py` et ouvrez-le.
2. Copiez **exactement** le bloc suivant, qui doit commencer à la première ligne du fichier.
3. Enregistrez avec `Ctrl + S`, puis exécutez la commande de vérification.

**LIGNES 1 À 14 - EN-TÊTE DE donnees.py**

```
"""Preparation, filtrage et indicateurs de ventes Atlas Commerce."""
from pathlib import Path
import pandas as pd

CHEMIN = Path(__file__).resolve().parent / "sales_data_brut.csv"
BRUT = pd.read_csv(CHEMIN, encoding="utf-8-sig")
REQUIS = [
    "Ligne_ID", "Commande_ID", "Date", "Produit", "Categorie",
    "Region", "Quantite", "Prix_Unitaire", "Cout_Unitaire",
    "Remise_Pct", "Chiffre_Affaires",
]
absentes = set(REQUIS) - set(BRUT.columns)
if absentes:
    raise ValueError(f"Colonnes manquantes : {sorted(absentes)}")
```

```
python -c "from donnees import BRUT; print(BRUT.shape)"
```

> **RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**
>
> Résultat `(4784, 11)` : le CSV a bien été chargé avec 4 784 lignes et 11 colonnes.

## Insertion 2/5 - corriger les types, supprimer les lignes invalides, calculer la marge

1. Placez le curseur à la fin du bloc précédent dans `donnees.py`.
2. Ajoutez **à la suite** le bloc ci-dessous. Attention, chaque ligne doit rester alignée comme montré.
3. Enregistrez, puis exécutez la commande de vérification.

**LIGNES 16 À 46 - NETTOYAGE ET STATISTIQUES DE QUALITÉ**

Code - partie 1/2 (coller les parties à la suite)

```
# Chaque ligne de transaction doit avoir un identifiant distinct.
df = BRUT.drop_duplicates(subset="Ligne_ID").copy()
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
for colonne in ["Quantite", "Prix_Unitaire", "Cout_Unitaire", "Remise_Pct", "Chiffre_Affaires"]:
    df[colonne] = pd.to_numeric(df[colonne], errors="coerce")

# Exclure les donnees inutilisables pour les indicateurs.
df = df.dropna(subset=REQUIS).copy()
df = df[
    (df["Quantite"] > 0)
    & (df["Prix_Unitaire"] > 0)
```

```
    & (df["Cout_Unitaire"] >= 0)
    & (df["Remise_Pct"].between(0, 1))
].copy()

# Controle de coherence entre quantite, prix, remise et CA.
ca_recalcule = (
    df["Quantite"] * df["Prix_Unitaire"] * (1 - df["Remise_Pct"])
).round(2)
df = df[(df["Chiffre_Affaires"] - ca_recalcule).abs() <= 0.01].copy()

# Marge commerciale simplifiee hors logistique, taxes et charges.
df["Marge"] = df["Chiffre_Affaires"] - df["Quantite"] * df["Cout_Unitaire"]
df = df.sort_values("Date").reset_index(drop=True)

STAT_QUALITE = {
    "brutes": len(BRUT),
    "doublons": int(BRUT["Ligne_ID"].duplicated().sum()),
    "retenues": len(df),
    "exclues_total": len(BRUT) - len(df),
```

Code - partie 2/2 (coller les parties à la suite)

```
}
```

```
python -c "from donnees import df, STAT_QUALITE; print(STAT_QUALITE)"
```

> **RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**
>
> Attendu : `{"brutes": 4784, "doublons": 1, "retenues": 4781, "exclues_total": 3}`.

D'abord le doublon de `Ligne_ID` est éliminé ; les valeurs mal typées sont converties, puis les lignes incomplètes ou incohérentes sont exclues. Une date illisible devient `NaT` grâce à `errors="coerce"`. La marge est calculée après l'opération de qualité.

## Insertion 3/5 - créer la fonction de filtrage

1. À la fin de `donnees.py`, sautez deux lignes.
2. Ajoutez `def filtrer(...)` avec le bloc ci-dessous.
3. Enregistrez et essayez le filtre sur une catégorie avec la commande de vérification.

**FONCTION filtrer()**

```
def filtrer(date_debut=None, date_fin=None, categorie="Toutes"):
    """Renvoie des lignes correspondant a la periode et a la categorie."""
    donnees = df
    if date_debut:
        donnees = donnees[donnees["Date"] >= pd.to_datetime(date_debut)]
    if date_fin:
        donnees = donnees[donnees["Date"] <= pd.to_datetime(date_fin)]
    if categorie and categorie != "Toutes":
        donnees = donnees[donnees["Categorie"] == categorie]
    return donnees.copy()
```

```
python -c "from donnees import filtrer; print(filtrer(categorie='Mode').shape)"
```

> **RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**
>
> La commande affiche un couple (nombre de lignes Mode, 12). Le premier nombre est inférieur à 4 781 et le second correspond aux 12 colonnes après création de `Marge`.

## Insertion 4/5 - calculer quatre KPI et le taux de marque

1. Toujours à la fin de `donnees.py`, sautez deux lignes.
2. Ajoutez la fonction `calculer_kpis()`.

3. Observez le `nunique()` sur `Commande_ID` : c'est la raison pour laquelle le nombre de commandes ne sera pas 4 781.

**FONCTION calculer_kpis()**

```
def calculer_kpis(donnees):
    """Calcule CA, commandes distinctes, panier, marge, taux de marque."""
    ca = float(donnees["Chiffre_Affaires"].sum())
    nb = int(donnees["Commande_ID"].nunique())
    marge = float(donnees["Marge"].sum())
    return {
        "ca": ca,
        "commandes": nb,
        "panier": ca / nb if nb else 0.0,
        "marge": marge,
        "taux_marque": 100 * marge / ca if ca else 0.0,
    }
```

## Insertion 5/5 - formater les montants et vérifier le fichier

1. Après la fonction précédente, insérez deux lignes vides.
2. Ajoutez `mad()` et le bloc de diagnostic `if __name__`.
3. Enregistrez puis lancez `python donnees.py` dans le terminal.

**FORMATAGE ET DIAGNOSTIC**

```
def mad(valeur):
    """Formate un montant en dirhams sans centimes."""
    return f"{valeur:,.0f}".replace(",", " ") + " MAD"


if __name__ == "__main__":
    print("QUALITE", STAT_QUALITE)
    print("TYPES\n", df.dtypes)
    print("KPI\n", calculer_kpis(df))
```

```
python donnees.py
python audit_donnees.py
```

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Le terminal affiche 4 781 lignes retenues, 3 exclues, 2 400 commandes et un CA de 7 671 306 MAD. Le contrôle `audit_donnees.py` valide 9 assertions.

**QUESTION / MINI-DÉFI**

Défi : dans `calculer_kpis`, remplacez temporairement `nunique()` par `count()` (sans sauvegarder votre version finale). Que devient le panier moyen ? Pourquoi ce calcul serait-il faux ? Remettez immédiatement `nunique()`.

**SI CELA NE MARCHE PAS**

Si `ImportError: cannot import name filtrer` apparaît, vérifiez que les cinq insertions ont été faites dans l'ordre et que les fonctions ne sont pas accidentellement indentées à l'intérieur d'un autre bloc.

MISSION 3.1-C · 15 MIN · FICHIER : DONNEES.PY

# Maîtriser les fonctions métier

**Votre objectif**

Pouvoir prédire les résultats d'un filtre et calculer les quatre KPI.

**Livrable visible :** Les KPI globaux de référence sont retrouvés.

1. Toujours dans `donnees.py`, cherchez `def filtrer(`.
2. Lisez successivement le filtre sur la date de début, la date de fin et la catégorie.
3. Cherchez `def calculer_kpis(` et repérez le `nunique()` de la commande.
4. Créez le fichier `controle_kpi.py` et entrez les quatre instructions suivantes.

```
from donnees import df, filtrer, calculer_kpis

kpis = calculer_kpis(df)
print("KPI globaux :", kpis)
print("Mode seule :", calculer_kpis(filtrer(categorie="Mode")))
print("Mois de janvier :", len(filtrer("2025-01-01", "2025-01-31")))
```

```
python controle_kpi.py
```

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Sur toutes les données retenues : **CA = 7 671 306 MAD**, **commandes distinctes = 2 400**, **panier moyen ≈ 3 196,38 MAD**, **marge commerciale = 2 607 006 MAD**.

Point métier : avec le filtre *Mode*, le panier moyen est le chiffre d'affaires des lignes Mode divisé par le nombre de commandes contenant au moins une ligne Mode. Ce n'est pas le panier complet de ces mêmes commandes.

**QUESTION / MINI-DÉFI**

Sans calculatrice, annoncez si le panier moyen sera supérieur ou inférieur à zéro lorsque la période ne contient aucune vente. Vérifiez ensuite comment la fonction gère ce cas.

MISSION 00 · 5 MIN · FICHIER : APP.PY

# Ouvrir le squelette au bon endroit

**Votre objectif**

Passer d'un fichier vide à un programme exécutable structuré.

**Livrable visible :** Un serveur Dash démarre avec une page encore vide.

1. Copiez `app_depart.py` à la racine en le renommant `app.py`. Si une solution `app.py` existe déjà dans le ZIP, renommez-la d'abord `app_solution.py`.
2. Ouvrez le nouveau `app.py`.
3. Repérez le bloc `app.layout = dbc.Container([ ... ])` et les commentaires `# ZONE_...`. Ces commentaires sont des **repères d'insertion** : ne les supprimez pas.
4. Exécutez `python app.py` et ouvrez l'adresse locale.

**SQUELETTE DE DÉPART - vérifiez que votre fichier lui ressemble**

Code - partie 1/2 (coller les parties à la suite)

```
"""Application Dash Atlas Commerce. Completer les ZONES dans l'ordre du TP."""
from dash import Dash, dcc, html, Input, Output, ctx
import dash_bootstrap_components as dbc
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from donnees import df, filtrer, calculer_kpis, mad

app = Dash(__name__, external_stylesheets=[dbc.themes.FLATLY])
app.title = "Atlas Commerce | Dashboard decisionnel"
DATE_MIN = df["Date"].min().date().isoformat()
DATE_MAX = df["Date"].max().date().isoformat()

# ZONE_FONCTIONS

app.layout = dbc.Container([
    # ZONE_ENTETE
    # ZONE_FILTRES
    # ZONE_KPIS
    # ZONE_EVOLUTION_UI
    # ZONE_CATEGORIES_UI
    # ZONE_REGIONS_UI
    # ZONE_MARGES_UI
    # ZONE_TOP_UI
    # ZONE_RESET_UI
    # ZONE_MESSAGE_UI
], fluid=True, className="px-3 px-xl-5 py-4")

# ZONE_KPI_CALLBACK
# ZONE_EVOLUTION_CB
```

Code - partie 2/2 (coller les parties à la suite)

```
# ZONE_CATEGORIES_CB
# ZONE_REGIONS_CB
# ZONE_MARGES_CB
# ZONE_TOP_CB
# ZONE_RESET_CB
# ZONE_MESSAGE_CB

if __name__ == "__main__":
    app.run(debug=True)
```

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Le serveur démarre sans erreur. Le navigateur présente une page presque vide ; aucun graphique n'est encore défini.

**SI CELA NE MARCHE PAS**

Si le terminal affiche `No module named donnees`, vérifiez que `app.py` et `donnees.py` sont au même niveau. Si le navigateur ne répond pas, vérifiez l'adresse et le port affichés par le terminal.

MISSION 01 · 5 MIN · FICHIER : APP.PY

# Construire l’en-tête du dashboard

**Votre objectif**

Afficher l’identité visuelle du client.

**Livrable visible :** Un bandeau « ATLAS COMMERCE » est visible.

1. Dans VS Code, ouvrez le fichier `app.py`, puis utilisez `Ctrl + F`.
2. Cherchez exactement `# ZONE_ENTETE` ; conservez cette ligne de commentaire.
3. **À l’intérieur de la liste de composants du `dbc.Container`,** Copiez le code suivant **juste sous** le marqueur. Si le code est scindé en parties, collez les parties l’une après l’autre.
4. Enregistrez avec `Ctrl + S`. Si le serveur a été arrêté, relancez `python app.py`.

**À AJOUTER SOUS # ZONE_ENTETE**

```
    html.Div([
        html.H1("ATLAS COMMERCE"),
        html.P("Pilotage des ventes | jeu de donnees fictif")
    ], className="bandeau"),
```

**Pourquoi cela fonctionne :** `html.Div` contient le titre `html.H1` et une phrase. La classe CSS `bandeau` est définie dans `assets/style.css`.

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l’obtenir**

Un bandeau foncé avec titre et sous-titre remplace la page vide.

**QUESTION / MINI-DÉFI**

Changez uniquement la phrase sous le titre et observez la mise à jour.

**SI CELA NE MARCHE PAS**

Si la classe CSS n’agit pas, vérifiez la présence du dossier `assets` exactement à la racine.

Secours : `checkpoints/01_*.py` (la numérotation correspond au jalon réalisé).

MISSION 02 · 30 MIN · FICHIER : APP.PY

# Ajouter trois filtres lisibles

**Votre objectif**

Créer les contrôles qui seront utilisés par les callbacks.

**Livrable visible :** Un calendrier, un Dropdown et un choix Jour/Mois.

1. Dans VS Code, ouvrez le fichier `app.py`, puis utilisez `Ctrl + F`.
2. Cherchez exactement `# ZONE_FILTRES` ; conservez cette ligne de commentaire.
3. **Toujours dans `app.layout`,** Copiez le code suivant **juste sous** le marqueur. Si le code est scindé en parties, collez les parties l'une après l'autre.
4. Enregistrez avec `Ctrl + S`. Si le serveur a été arrêté, relancez `python app.py`.

**À AJOUTER SOUS # ZONE_FILTRES**

Code - partie 1/2 (coller les parties à la suite)

```
    dbc.Card(
        dbc.CardBody(
            dbc.Row([
                dbc.Col([
                    html.Label("Periode", className="etiquette"),
                    dcc.DatePickerRange(
                        id="dates", start_date=DATE_MIN,
                        end_date=DATE_MAX,
                        min_date_allowed=DATE_MIN,
                        max_date_allowed=DATE_MAX,
                        display_format="DD/MM/YYYY"
                    )
                ], xs=12, lg=5),
                dbc.Col([
                    html.Label("Categorie", className="etiquette"),
                    dcc.Dropdown(
                        id="categorie", value="Toutes",
                        options=["Toutes"] + sorted(df["Categorie"].unique()),
                        clearable=False
                    )
                ], xs=12, lg=4),
                dbc.Col([
                    html.Label("Regroupement temporel", className="etiquette"),
                    dcc.RadioItems(
                        id="freq", value="M", inline=True,
                        options=[
                            {"label": "Jour", "value": "J"},
                            {"label": "Mois", "value": "M"}
                        ]
                    )
```

Code - partie 2/2 (coller les parties à la suite)

```
                ], xs=12, lg=3),
            ], className="g-3")
        ), className="filter-card mb-3"
    ),
```

**Pourquoi cela fonctionne :** `dbc.Row` contient des `dbc.Col`. `xs=12` signifie pleine largeur sur petits écrans ; `lg=5` signifie 5/12 colonnes à grande largeur. Les identifiants des filtres seront réutilisés dans les callbacks.

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Les trois contrôles sont visibles. À ce stade, leurs changements ne modifient aucun indicateur : c'est normal.

**QUESTION / MINI-DÉFI**

Citez les trois identifiants des filtres et la propriété qui sera surveillée pour chacun.

**SI CELA NE MARCHE PAS**

Si un sélecteur ne s’affiche pas, vérifiez les virgules entre les composants du layout.

Secours : `checkpoints/02_*.py` (la numérotation correspond au jalon réalisé).

MISSION 03 · 25 MIN · FICHIER : APP.PY

# Créer les cartes KPI non connectées

**Votre objectif**

Afficher quatre cases, puis comprendre l'utilité d'une fonction réutilisable.

**Livrable visible :** Quatre cartes vides (--) se placent sur une ligne ou se replient sur petits écrans.

1. Dans VS Code, ouvrez le fichier `app.py`, puis utilisez `Ctrl + F`.
2. Cherchez exactement `# ZONE_FONCTIONS` ; conservez cette ligne de commentaire.
3. **Attention : ce marqueur est avant `app.layout`, et non dans sa liste.** Copiez le code suivant **juste sous** le marqueur. Si le code est scindé en parties, collez les parties l'une après l'autre.
4. Enregistrez avec `Ctrl + S`. Si le serveur a été arrêté, relancez `python app.py`.

**À AJOUTER SOUS # ZONE_FONCTIONS**

Code - partie 1/2 (coller les parties à la suite)

```
def carte_kpi(titre, identifiant):
    """Construit une carte KPI reutilisable."""
    return dbc.Card(dbc.CardBody([
        html.Div(titre, className="kpi-label"),
        html.Div("--", id=identifiant, className="kpi-valeur")
    ]), className="kpi-card")


def carte_graphique(identifiant):
    """Construit une carte contenant un graphique Dash."""
    return dbc.Card(dbc.CardBody(
        dcc.Loading(dcc.Graph(id=identifiant))
    ), className="graph-card")


def figure_vide(titre):
    """Affiche un message pour les filtres sans ventes."""
    fig = go.Figure()
    fig.add_annotation(
        text="Aucune vente pour ces filtres",
        x=0.5, y=0.5, xref="paper", yref="paper",
        showarrow=False
    )
    fig.update_xaxes(visible=False)
    fig.update_yaxes(visible=False)
    fig.update_layout(title=titre, template="plotly_white", height=350)
    return fig


def harmoniser(fig):
```

Code - partie 2/2 (coller les parties à la suite)

```
    """Applique une hauteur et un fond communs aux figures."""
    fig.update_layout(
        template="plotly_white", height=360,
        margin=dict(l=25, r=20, t=60, b=35)
    )
    return fig
```

**Pourquoi cela fonctionne :** `carte_kpi(titre, identifiant)` factorise les cartes. `figure_vide` et `harmoniser` serviront plus tard à uniformiser les figures.

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Le programme fonctionne mais les cartes ne sont pas encore visibles : il faut effectuer l'ajout suivant dans `# ZONE_KPIS`.

**QUESTION / MINI-DÉFI**

Expliquez pourquoi nous avons défini la fonction `carte_kpi` au lieu de recopier tout le code d'une carte quatre fois.

Secours : `checkpoints/03_*.py` (la numérotation correspond au jalon réalisé).

## MISSION 03 - deuxième insertion dans la même étape

1. Dans `app.py`, recherchez `# ZONE_KPIS`.
2. Copiez le bloc ci-dessous immédiatement après ce commentaire.
3. Enregistrez et regardez le navigateur.

**SOUS # ZONE_KPIS**

```
    dbc.Row([
        dbc.Col(carte_kpi("Chiffre d'affaires", "ca"), xs=12, sm=6, lg=3),
        dbc.Col(carte_kpi("Commandes distinctes", "commandes"), xs=12, sm=6, lg=3),
        dbc.Col(carte_kpi("Panier moyen", "panier"), xs=12, sm=6, lg=3),
        dbc.Col(carte_kpi("Marge commerciale", "marge"), xs=12, sm=6, lg=3),
    ], className="g-3 mb-4"),
    html.P(id="etat", className="text-muted"),
```

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Vous voyez quatre cartes : Chiffre d'affaires, Commandes distinctes, Panier moyen, Marge commerciale. Chaque carte affiche encore `--`.

**QUESTION / MINI-DÉFI**

Redimensionnez le navigateur : combien de cartes sont affichées par ligne en grand écran puis en petit écran ?

MISSION 04 · 45 MIN · FICHIER : APP.PY

# Rendre les indicateurs vraiment dynamiques

**Votre objectif**

Créer le premier callback métier d'Atlas Commerce.

**Livrable visible :** Les quatre KPI et le nombre de lignes réagissent aux filtres.

1. Dans VS Code, ouvrez le fichier `app.py`, puis utilisez `Ctrl + F`.
2. Cherchez exactement `# ZONE_KPI_CALLBACK` ; conservez cette ligne de commentaire.
3. **Ici, le marqueur se trouve après la fermeture de `app.layout` : n'insérez pas le décorateur dans la liste des composants.** Copiez le code suivant **juste sous** le marqueur. Si le code est scindé en parties, collez les parties l'une après l'autre.
4. Enregistrez avec `Ctrl + S`. Si le serveur a été arrêté, relancez `python app.py`.

**À AJOUTER SOUS # ZONE_KPI_CALLBACK**

```
@app.callback(
    Output("ca", "children"),
    Output("commandes", "children"),
    Output("panier", "children"),
    Output("marge", "children"),
    Output("etat", "children"),
    Input("dates", "start_date"),
    Input("dates", "end_date"),
    Input("categorie", "value")
)
def actualiser_kpis(debut, fin, categorie):
    ventes = filtrer(debut, fin, categorie)
    k = calculer_kpis(ventes)
    message = f"{len(ventes)} lignes | {k['commandes']} commandes"
    return (
        mad(k["ca"]), str(k["commandes"]),
        mad(k["panier"]), mad(k["marge"]), message
    )
```

**Pourquoi cela fonctionne :** Trois entrées de filtre déterminent cinq sorties. Le résultat de `calculer_kpis(ventes)` vient de pandas. Les valeurs retournées correspondent, dans le même ordre, aux cinq `Output`.

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Sur toutes les données : 7 671 306 MAD de CA, 2 400 commandes, 3 196 MAD environ de panier et 2 607 006 MAD de marge. Après sélection de « Mode », ces valeurs changent.

**QUESTION / MINI-DÉFI**

Sélectionnez « Maison » puis revenez sur « Toutes » ; pourquoi le nombre de lignes ne correspond-il pas au nombre de commandes ?

**SI CELA NE MARCHE PAS**

Si Dash affiche une erreur `NonExistentIdException`, revérifiez les cinq IDs présents dans les cartes du layout.

Secours : `checkpoints/04_*.py` (la numérotation correspond au jalon réalisé).

MISSION 05 · 20 MIN · FICHIER : APP.PY

# Afficher la courbe du chiffre d’affaires

**Votre objectif**

Relier la granularité Jour/Mois à une courbe Plotly.

**Livrable visible :** Une courbe du CA avec points et survol interactif.

## Action A - créer l’emplacement dans la page

1. Dans `app.py`, appuyez sur `Ctrl + F` et recherchez `# ZONE_EVOLUTION_UI`.
2. Ajoutez juste sous le marqueur le bloc du layout ci-dessous. **Ne le mettez pas après le `if __name__`.**
3. Enregistrez. La zone graphique peut apparaître vide tant que la fonction ne fournit pas encore de figure.

**À COLLER SOUS # ZONE_EVOLUTION_UI**

```
    dbc.Row([
        dbc.Col(carte_graphique("evolution"), xs=12)
    ], className="g-3 mb-4"),
```

## Action B - relier les données au composant

1. Cherchez maintenant `# ZONE_EVOLUTION_CB`, situé **après la fermeture de `app.layout`**.
2. Collez juste sous ce deuxième marqueur le callback complet ci-dessous.
3. Sauvegardez, revenez dans le navigateur et appliquez plusieurs filtres.

**À COLLER SOUS # ZONE_EVOLUTION_CB**

```
@app.callback(
    Output("evolution", "figure"),
    Input("dates", "start_date"),
    Input("dates", "end_date"),
    Input("categorie", "value"),
    Input("freq", "value")
)
def tracer_evolution(debut, fin, categorie, freq):
    ventes = filtrer(debut, fin, categorie)
    if ventes.empty:
        return figure_vide("Evolution du chiffre d'affaires")
    ventes["Periode"] = (
        ventes["Date"].dt.to_period("M").dt.to_timestamp()
        if freq == "M" else ventes["Date"]
    )
    evolution = (ventes.groupby("Periode", as_index=False)
                 ["Chiffre_Affaires"].sum().sort_values("Periode"))
    fig = px.line(
        evolution, x="Periode", y="Chiffre_Affaires",
        markers=True, title="Evolution du chiffre d'affaires",
        labels={"Chiffre_Affaires": "CA (MAD)"}
    )
    fig.update_traces(line_color="#2866aa")
    return harmoniser(fig)
```

**Compréhension :** Le regroupement se fait avec `groupby("Periode")`. Pour un mois, `dt.to_period("M").dt.to_timestamp()` transforme chaque date en premier jour du mois correspondant, puis le CA est additionné.

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l’obtenir**

Le graphique apparaît. Le passage « Mois » → « Jour » change le nombre de points sans changer le total du CA.

**QUESTION / MINI-DÉFI**

Pourquoi faut-il utiliser `sum()` plutôt que `mean()` pour l'évolution du CA total ?

**SI CELA NE MARCHE PAS**

Si la carte du graphique existe mais reste vide, recherchez dans le terminal un traceback. Vérifiez aussi l'ID dans `dcc.Graph` et l'`Output(..., "figure")`.

Solution intermédiaire : `checkpoints/05_*.py`.

MISSION 06 · 15 MIN · FICHIER : APP.PY

# Comparer les catégories avec des barres

**Votre objectif**

Identifier la catégorie au CA le plus élevé.

**Livrable visible :** Un graphique en barres par catégorie.

## Action A - créer l'emplacement dans la page

1. Dans `app.py`, appuyez sur `Ctrl + F` et recherchez `# ZONE_CATEGORIES_UI`.
2. Ajoutez juste sous le marqueur le bloc du layout ci-dessous. **Ne le mettez pas après le `if __name__`.**
3. Enregistrez. La zone graphique peut apparaître vide tant que la fonction ne fournit pas encore de figure.

**À COLLER SOUS # ZONE_CATEGORIES_UI**

```
    dbc.Row([
        dbc.Col(carte_graphique("categories"), xs=12)
    ], className="g-3 mb-4"),
```

## Action B - relier les données au composant

1. Cherchez maintenant `# ZONE_CATEGORIES_CB`, situé **après la fermeture de `app.layout`**.
2. Collez juste sous ce deuxième marqueur le callback complet ci-dessous.
3. Sauvegardez, revenez dans le navigateur et appliquez plusieurs filtres.

**À COLLER SOUS # ZONE_CATEGORIES_CB**

```
@app.callback(
    Output("categories", "figure"),
    Input("dates", "start_date"),
    Input("dates", "end_date"),
    Input("categorie", "value")
)
def tracer_categories(debut, fin, categorie):
    ventes = filtrer(debut, fin, categorie)
    if ventes.empty:
        return figure_vide("CA par categorie")
    resume = (ventes.groupby("Categorie", as_index=False)
              ["Chiffre_Affaires"].sum()
              .sort_values("Chiffre_Affaires", ascending=False))
    fig = px.bar(
        resume, x="Categorie", y="Chiffre_Affaires",
        color="Categorie", title="CA par categorie"
    )
    fig.update_layout(showlegend=False)
    return harmoniser(fig)
```

**Compréhension :** Pandas crée une ligne agrégée par catégorie. Plotly transforme le tableau agrégé en barres ; le callback réagit aux trois filtres généraux.

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Le graphique montre les catégories ; après le filtre « Mode », il ne reste qu'une barre.

**QUESTION / MINI-DÉFI**

Expliquez à un manager pourquoi il faut lire l'axe vertical avant de comparer les barres.

**SI CELA NE MARCHE PAS**

Si la carte du graphique existe mais reste vide, recherchez dans le terminal un traceback. Vérifiez aussi l'ID dans `dcc.Graph` et l'`Output(..., "figure")`.

Solution intermédiaire : `checkpoints/06_*.py`.

MISSION 07 · 15 MIN · FICHIER : APP.PY

# Afficher la répartition géographique

**Votre objectif**

Visualiser le poids relatif des régions dans le chiffre d'affaires.

**Livrable visible :** Un donut à six régions.

## Action A - créer l'emplacement dans la page

1. Dans `app.py`, appuyez sur `Ctrl + F` et recherchez `# ZONE_REGIONS_UI`.
2. Ajoutez juste sous le marqueur le bloc du layout ci-dessous. **Ne le mettez pas après le `if __name__`.**
3. Enregistrez. La zone graphique peut apparaître vide tant que la fonction ne fournit pas encore de figure.

**À COLLER SOUS # ZONE_REGIONS_UI**

```
    dbc.Row([
        dbc.Col(carte_graphique("regions"), xs=12)
    ], className="g-3 mb-4"),
```

## Action B - relier les données au composant

1. Cherchez maintenant `# ZONE_REGIONS_CB`, situé **après la fermeture de `app.layout`**.
2. Collez juste sous ce deuxième marqueur le callback complet ci-dessous.
3. Sauvegardez, revenez dans le navigateur et appliquez plusieurs filtres.

**À COLLER SOUS # ZONE_REGIONS_CB**

```
@app.callback(
    Output("regions", "figure"),
    Input("dates", "start_date"),
    Input("dates", "end_date"),
    Input("categorie", "value")
)
def tracer_regions(debut, fin, categorie):
    ventes = filtrer(debut, fin, categorie)
    if ventes.empty:
        return figure_vide("Part de chaque region")
    resume = (ventes.groupby("Region", as_index=False)
              ["Chiffre_Affaires"].sum())
    fig = px.pie(
        resume, names="Region", values="Chiffre_Affaires",
        title="Part de chaque region dans le CA", hole=0.40
    )
    fig.update_traces(textinfo="percent")
    return harmoniser(fig)
```

**Compréhension :** Le diagramme montre des *parts de CA*, non la position géographique réelle des régions. La propriété Plotly `clickData` sera utilisée dans la mission 10.

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Sur « Toutes », six secteurs sont présents. Le survol affiche la région et le montant associé.

**QUESTION / MINI-DÉFI**

Cherchez la région avec la contribution la plus importante. Est-ce nécessairement la région avec la meilleure marge ?

**SI CELA NE MARCHE PAS**

Si la carte du graphique existe mais reste vide, recherchez dans le terminal un traceback. Vérifiez aussi l'ID dans `dcc.Graph` et l'`Output(..., "figure")`.

Solution intermédiaire : `checkpoints/07_*.py`.

MISSION 08 · 15 MIN · FICHIER : APP.PY

# Comparer les marges commerciales

**Votre objectif**

Éviter de confondre ventes élevées et rentabilité.

**Livrable visible :** Un graphique des marges par catégorie.

## Action A - créer l'emplacement dans la page

1. Dans `app.py`, appuyez sur `Ctrl + F` et recherchez `# ZONE_MARGES_UI`.
2. Ajoutez juste sous le marqueur le bloc du layout ci-dessous. **Ne le mettez pas après le `if __name__`.**
3. Enregistrez. La zone graphique peut apparaître vide tant que la fonction ne fournit pas encore de figure.

**À COLLER SOUS # ZONE_MARGES_UI**

```
    dbc.Row([
        dbc.Col(carte_graphique("marges"), xs=12)
    ], className="g-3 mb-4"),
```

## Action B - relier les données au composant

1. Cherchez maintenant `# ZONE_MARGES_CB`, situé **après la fermeture de `app.layout`**.
2. Collez juste sous ce deuxième marqueur le callback complet ci-dessous.
3. Sauvegardez, revenez dans le navigateur et appliquez plusieurs filtres.

**À COLLER SOUS # ZONE_MARGES_CB**

```
@app.callback(
    Output("marges", "figure"),
    Input("dates", "start_date"),
    Input("dates", "end_date"),
    Input("categorie", "value")
)
def tracer_marges(debut, fin, categorie):
    ventes = filtrer(debut, fin, categorie)
    if ventes.empty:
        return figure_vide("Marge par categorie")
    resume = (ventes.groupby("Categorie", as_index=False)
              ["Marge"].sum().sort_values("Marge", ascending=False))
    fig = px.bar(
        resume, x="Categorie", y="Marge", color="Categorie",
        title="Marge commerciale par categorie"
    )
    fig.update_layout(showlegend=False)
    return harmoniser(fig)
```

**Compréhension :** La marge a été calculée ligne par ligne lors du nettoyage. Le graphique somme ensuite ces marges par catégorie ; il ne représente pas la marge nette.

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Les barres de marge apparaissent ; elles changent avec les dates et la catégorie.

**QUESTION / MINI-DÉFI**

La catégorie au CA le plus élevé est-elle automatiquement celle qui a le meilleur taux de marque ? Justifiez sans inventer de données.

**SI CELA NE MARCHE PAS**

Si la carte du graphique existe mais reste vide, recherchez dans le terminal un traceback. Vérifiez aussi l'ID dans `dcc.Graph` et l'`Output(..., "figure")`.

Solution intermédiaire : `checkpoints/08_*.py`.

MISSION 09 · 35 MIN · FICHIER : APP.PY

# Construire le Top 10 produits horizontal

> **Votre objectif**
>
> Classer les produits selon le CA, tout en préparant le filtre croisé.
>
> **Livrable visible :** Un Top 10 horizontal et un titre contextuel.

## Action A - créer l'emplacement dans la page

1. Dans `app.py`, appuyez sur `Ctrl + F` et recherchez `# ZONE_TOP_UI`.
2. Ajoutez juste sous le marqueur le bloc du layout ci-dessous. **Ne le mettez pas après le `if __name__`.**
3. Enregistrez. La zone graphique peut apparaître vide tant que la fonction ne fournit pas encore de figure.

**À COLLER SOUS # ZONE_TOP_UI**

```
    dbc.Row([
        dbc.Col([
            html.H5(id="titre-top"),
            carte_graphique("top-produits")
        ], xs=12)
    ], className="g-3 mb-4"),
    dcc.Store(id="region-active", data=None),
```

## Action B - relier les données au composant

1. Cherchez maintenant `# ZONE_TOP_CB`, situé **après la fermeture de `app.layout`**.
2. Collez juste sous ce deuxième marqueur le callback complet ci-dessous.
3. Sauvegardez, revenez dans le navigateur et appliquez plusieurs filtres.

**À COLLER SOUS # ZONE_TOP_CB**

```
@app.callback(
    Output("top-produits", "figure"),
    Output("titre-top", "children"),
    Input("dates", "start_date"),
    Input("dates", "end_date"),
    Input("categorie", "value"),
    Input("region-active", "data")
)
def tracer_top(debut, fin, categorie, region):
    ventes = filtrer(debut, fin, categorie)
    if region:
        ventes = ventes[ventes["Region"] == region].copy()
    titre = "Top produits" + (f" | {region}" if region else " | Toutes regions")
    if ventes.empty:
        return figure_vide(titre), titre
    resume = (ventes.groupby("Produit", as_index=False)
              ["Chiffre_Affaires"].sum()
              .nlargest(10, "Chiffre_Affaires")
              .sort_values("Chiffre_Affaires"))
    fig = px.bar(
        resume, x="Chiffre_Affaires", y="Produit",
        orientation="h", color="Chiffre_Affaires",
        color_continuous_scale="Blues"
    )
    fig.update_layout(coloraxis_showscale=False)
    return harmoniser(fig), titre
```

**Compréhension :** `nlargest(10)` retient dix produits puis `sort_values()` prépare l'ordre d'affichage horizontal. `dcc.Store(id="region-active")` mémorisera plus tard la région cliquée.

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Les produits apparaissent. Le titre dit « Toutes regions » car aucun clic n'a encore été mémorisé.

**QUESTION / MINI-DÉFI**

Le graphique du Top 10, avec des produits à prix unitaire très différents, mesure-t-il des volumes ou un chiffre d'affaires ?

**SI CELA NE MARCHE PAS**

Si la carte du graphique existe mais reste vide, recherchez dans le terminal un traceback. Vérifiez aussi l'ID dans `dcc.Graph` et l'`Output(..., "figure")`.

Solution intermédiaire : `checkpoints/09_*.py`.

MISSION 10 · 45 MIN · FICHIER : APP.PY

# Cliquer sur une région pour filtrer les produits

**Votre objectif**

Connecter deux graphiques par un événement, avec reset explicite.

**Livrable visible :** Le Top 10 réagit au clic sur un secteur ; un bouton annule le filtre.

## Action A - ajouter un bouton de remise à zéro

1. Dans `app.py`, localisez `# ZONE_RESET_UI` dans le layout.
2. Collez le bloc du bouton immédiatement en dessous.

**SOUS # ZONE_RESET_UI**

```
    dbc.Row([
        dbc.Col(dbc.Button(
            "Effacer le filtre region", id="reset-region",
            color="outline-secondary", className="mb-3"
        ), xs=12)
    ]),
```

## Action B - enregistrer l'interaction dans dcc.Store

1. Repérez `# ZONE_RESET_CB` **après** le layout.
2. Collez la fonction suivante sous le marqueur ; `ctx.triggered_id` indique la cause de l'appel.
3. Enregistrez, puis dans le navigateur, cliquez sur un secteur du donut ; regardez le titre du Top 10.
4. Cliquez sur *Effacer le filtre région* : le Top 10 doit revenir à toutes les régions.

**SOUS # ZONE_RESET_CB**

```
@app.callback(
    Output("region-active", "data"),
    Input("regions", "clickData"),
    Input("reset-region", "n_clicks"),
    Input("dates", "start_date"),
    Input("dates", "end_date"),
    Input("categorie", "value"),
    prevent_initial_call=True
)
def selectionner_region(clic, _reset, _debut, _fin, _categorie):
    """Memorise le clic regional ; sinon efface le filtre local."""
    if ctx.triggered_id == "regions" and clic and clic.get("points"):
        return clic["points"][0].get("label")
    return None
```

Le diagramme régional produit une information lors du clic. Le callback écrit cette région dans `dcc.Store`. Le callback Top 10, déjà créé, lit ce stockage et filtre ses lignes. **Les KPI globaux et les autres graphiques ne sont pas filtrés par ce clic régional** : seul le Top 10 l'est. C'est une convention explicite de ce TP.

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l'obtenir**

Cliquez par exemple sur « Souss-Massa » : le titre du Top 10 mentionne cette région. Cliquez ensuite sur le bouton de remise à zéro : le titre revient à toutes les régions.

**QUESTION / MINI-DÉFI**

Quel est l'avantage de stocker le nom de la région dans `dcc.Store` plutôt que de faire dépendre toute la logique du Top 10 directement de `clickData` ?

**SI CELA NE MARCHE PAS**

Sur un donut Plotly, le nom d'un secteur est souvent disponible dans `clickData["points"][0]["label"]`. Si vous obtenez `None`, utilisez un `print(clic)` dans le callback pour inspecter les valeurs réellement reçues.

MISSION 11 · 15 MIN · FICHIER : APP.PY

# Ajouter une phrase d’analyse métier

**Votre objectif**

Transformer un dashboard descriptif en aide à la lecture.

**Livrable visible :** Une alerte affiche la région au CA le plus haut et la catégorie à la marge la plus élevée.

## Action A - ajouter une zone de message

1. Cherchez `# ZONE_MESSAGE_UI` dans le layout.
2. Collez le bloc ci-dessous.

**SOUS # ZONE_MESSAGE_UI**

```
    dbc.Alert(id="lecture-metier", color="info"),
```

## Action B - calculer le commentaire à partir des filtres

1. Cherchez `# ZONE_MESSAGE_CB` sous le layout.
2. Ajoutez le callback suivant, enregistrez, puis testez une catégorie et une période restreinte.

**SOUS # ZONE_MESSAGE_CB**

```
@app.callback(
    Output("lecture-metier", "children"),
    Input("dates", "start_date"),
    Input("dates", "end_date"),
    Input("categorie", "value")
)
def recommandation_metier(debut, fin, categorie):
    ventes = filtrer(debut, fin, categorie)
    if ventes.empty:
        return "Aucune vente : elargissez la plage ou changez la categorie."
    par_region = ventes.groupby("Region")["Chiffre_Affaires"].sum()
    par_categorie = ventes.groupby("Categorie")["Marge"].sum()
    region = par_region.idxmax()
    cat = par_categorie.idxmax()
    return (f"Region au CA le plus eleve : {region}. "
            f"Categorie a la marge totale la plus forte : {cat}. "
            "Analyse sur les filtres courants, pas en temps reel.")
```

**RÉSULTAT ATTENDU - ne passez pas à la suite avant de l’obtenir**

Une phrase métier apparaît sous les graphiques, puis change lorsque les filtres généraux changent. Elle indique bien « Analyse sur les filtres courants, pas en temps réel ».

**QUESTION / MINI-DÉFI**

La région au CA le plus élevé est-elle forcément la région la plus rentable ? Quelles données ou quels calculs manquent pour conclure ?

# Recette finale - réaliser une démonstration sans assistance

Vous avez assemblé le dashboard par petites couches. Il reste à prouver que les calculs, les composants et les interactions sont cohérents.

| N° | Test | Résultat attendu |
|---|---|---|
| 1 | Ouvrir la page | Bandeau, 3 filtres, 4 KPI, 5 graphiques |
| 2 | Filtres « Toutes », période complète | CA = 7 671 306 MAD |
| 3 | Période complète | 2 400 commandes |
| 4 | Période complète | Marge = 2 607 006 MAD |
| 5 | Catégorie « Maison » | KPI + graphes changent ensemble |
| 6 | Mois → Jour | Seule la granularité de la courbe change |
| 7 | Clic sur un secteur régional | Top 10 filtré et titre modifié |
| 8 | Bouton reset région | Top 10 toutes régions |
| 9 | Dates sans ventes (si autorisées) | Figures sans données, KPI à zéro |
| 10 | Écran étroit | Cartes réorganisées, pas de chevauchement majeur |

## Tester les calculs, pas seulement l’interface

```
python tests_metier.py
python -m compileall app.py donnees.py
```

> **RÉSULTAT ATTENDU - ne passez pas à la suite avant de l’obtenir**
>
> Résultat attendu : `12 tests metier : OK`. La compilation Python ne doit signaler aucune erreur de syntaxe.

## Exporter les dépendances de votre venv

```
python -m pip freeze > requirements.txt
```

Le fichier `requirements.txt` est à transmettre avec votre projet. Pour installer les dépendances sur un autre PC, il faudra activer son propre environnement virtuel puis exécuter `python -m pip install -r requirements.txt`. On ne partage pas normalement le dossier `.venv`.

## Mini-soutenance de 5 minutes

1. Montrez les quatre KPI globaux et expliquez les définitions du panier moyen et de la marge.
2. Filtrez sur une catégorie de produits et décrivez la variation de la courbe du CA.
3. Cliquez sur une région du donut et observez le Top 10.
4. Expliquez quelles informations réelles manqueraient pour suivre la rentabilité nette.
5. Montrez le fichier de tests et expliquez pourquoi un dashboard « joli » peut être faux.

ANNEXES

# Comprendre, déboguer et aller plus loin

Les dernières pages servent de référence pendant la réalisation du projet et après la formation.

## Annexe A - erreurs typiques et leur diagnostic

| Symptôme | Où regarder en priorité |
|---|---|
| `ModuleNotFoundError: dash` | Interpréteur `.venv` actif ? Installation `python -m pip install ...` dans le bon terminal ? |
| `No module named donnees` | `app.py` et `donnees.py` dans le même dossier racine ? |
| Port 8050 déjà utilisé | Un autre script Dash tourne-t-il ? Arrêtez avec `Ctrl + C`. |
| Erreur de syntaxe près de la ligne collée | Virgule de fin dans le layout, parenthèses et indentation du bloc ? |
| `NonExistentIdException` | Un ID déclaré en `Input/Output` manque dans le layout. |
| Valeurs KPI bloquées à « -- » | Callback écrit au bon endroit ? IDs des Output corrects ? |
| Graphique vide | La figure est-elle retournée ? La période contient-elle des ventes ? |
| Montant incohérent | Sommez le CA net de remise ; compter les commandes avec `nunique()`. |
| Filtres non synchronisés | Chaque figure utilise-t-elle la même fonction `filtrer()` ? |
| CSS absent | Dossier `assets` à la racine ; nom `style.css` exact. |

### Technique de débogage : imprimer les informations du callback

```
print("Date de debut :", debut)
print("Date de fin :", fin)
print("Categorie :", categorie)
print("Dimensions apres filtrage :", ventes.shape)
print("Extrait :", ventes.head(3))
```

Collez temporairement ces impressions juste après `ventes = filtrer(...)` dans un callback. Ces messages apparaîtront dans le terminal de VS Code. Retirez-les une fois le diagnostic terminé.

## Annexe B - vocabulaire utile

| Terme | Sens |
|---|---|
| `layout` | Arbre des éléments à afficher sur la page |
| `children` | Contenu textuel ou composants enfants d'un élément |
| `Input` | Propriété déclenchant un callback |
| `Output` | Propriété modifiée par le callback |
| `dcc.Graph` | Composant Dash recevant une figure Plotly |
| `groupby` | Regroupement pandas pour réaliser des agrégations |
| `dbc.Row / dbc.Col` | Disposition responsive basée sur une grille Bootstrap |
| `dcc.Store` | Mémorisation de petites données côté navigateur |
| `clickData` | Informations d'un point cliqué dans un graphique |

## Annexe C - performance et évolutions de Dash 4.x

La version stable vérifiée au 23 octobre 2026 est Dash 4.4.1 ; Dash 4.5.0rc1 est une préversion. La famille Dash 4.x a modernisé les composants et l'accessibilité. Le composant utilisé dans ce guide reste `dcc.DatePickerRange` : il ne faut pas le remplacer par un nom supposé comme `dcc.DatePicker`.

Pour une petite application CSV, il suffit de charger les données une fois et d'utiliser pandas pour préparer les tableaux avant `px.line` ou `px.bar`. Pour de grands volumes, envisagez un cache côté serveur, par exemple `ServersideOutputTransform` et `Serverside` de dash-extensions ; pour les interactions légères, étudiez les clientside callbacks. Ces outils ne sont pas nécessaires pour réussir le parcours principal.

Depuis Dash 4.2, FastAPI et Quart sont disponibles en plus du backend Flask. Les **WebSocket callbacks** exigent FastAPI ou Quart et sont utiles lorsque le serveur doit pousser des mises à jour progressives, sans interroger périodiquement le serveur avec `dcc.Interval`. Le CSV de ce TP est statique : nous ne simulons pas une fausse application temps réel.

## Annexe D - sources documentaires consultées

- Dash : documentation officielle
- Dash : Basic Callbacks
- Dash : dcc.DatePickerRange
- Dash : Server Backends
- Dash : WebSocket Callbacks
- GitHub : releases Plotly Dash

# Annexe E - listing intégral de donnees.py

Ce script a été fourni dans le ZIP pour concentrer l'apprentissage sur Dash pendant les six heures du projet. Il reste intégralement disponible ici pour l'étudier, le modifier ou le reconstruire avec votre formateur.

**FICHIER COMPLET - donnees.py**

Code - partie 1/3 (coller les parties à la suite)

```
"""Preparation, filtrage et indicateurs de ventes Atlas Commerce."""
from pathlib import Path
import pandas as pd

CHEMIN = Path(__file__).resolve().parent / "sales_data_brut.csv"
```

```
BRUT = pd.read_csv(CHEMIN, encoding="utf-8-sig")
REQUIS = [
    "Ligne_ID", "Commande_ID", "Date", "Produit", "Categorie",
    "Region", "Quantite", "Prix_Unitaire", "Cout_Unitaire",
    "Remise_Pct", "Chiffre_Affaires",
]
absentes = set(REQUIS) - set(BRUT.columns)
if absentes:
    raise ValueError(f"Colonnes manquantes : {sorted(absentes)}")

# Chaque ligne de transaction doit avoir un identifiant distinct.
df = BRUT.drop_duplicates(subset="Ligne_ID").copy()
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
for colonne in ["Quantite", "Prix_Unitaire", "Cout_Unitaire", "Remise_Pct", "Chiffre_Affaires"]:
    df[colonne] = pd.to_numeric(df[colonne], errors="coerce")

# Exclure les donnees inutilisables pour les indicateurs.
df = df.dropna(subset=REQUIS).copy()
df = df[
    (df["Quantite"] > 0)
    & (df["Prix_Unitaire"] > 0)
    & (df["Cout_Unitaire"] >= 0)
    & (df["Remise_Pct"].between(0, 1))
].copy()
```

Code - partie 2/3 (coller les parties à la suite)

```
# Controle de coherence entre quantite, prix, remise et CA.
ca_recalcule = (
    df["Quantite"] * df["Prix_Unitaire"] * (1 - df["Remise_Pct"])
).round(2)
df = df[(df["Chiffre_Affaires"] - ca_recalcule).abs() <= 0.01].copy()

# Marge commerciale simplifiee hors logistique, taxes et charges.
df["Marge"] = df["Chiffre_Affaires"] - df["Quantite"] * df["Cout_Unitaire"]
df = df.sort_values("Date").reset_index(drop=True)

STAT_QUALITE = {
    "brutes": len(BRUT),
    "doublons": int(BRUT["Ligne_ID"].duplicated().sum()),
    "retenues": len(df),
    "exclues_total": len(BRUT) - len(df),
}


def filtrer(date_debut=None, date_fin=None, categorie="Toutes"):
    """Renvoie des lignes correspondant a la periode et a la categorie."""
    donnees = df
    if date_debut:
        donnees = donnees[donnees["Date"] >= pd.to_datetime(date_debut)]
    if date_fin:
        donnees = donnees[donnees["Date"] <= pd.to_datetime(date_fin)]
    if categorie and categorie != "Toutes":
        donnees = donnees[donnees["Categorie"] == categorie]
    return donnees.copy()
```

Code - partie 3/3 (coller les parties à la suite)

```
def calculer_kpis(donnees):
    """Calcule CA, commandes distinctes, panier, marge, taux de marque."""
    ca = float(donnees["Chiffre_Affaires"].sum())
    nb = int(donnees["Commande_ID"].nunique())
    marge = float(donnees["Marge"].sum())
    return {
        "ca": ca,
        "commandes": nb,
        "panier": ca / nb if nb else 0.0,
        "marge": marge,
        "taux_marque": 100 * marge / ca if ca else 0.0,
    }


def mad(valeur):
    """Formate un montant en dirhams sans centimes."""
    return f"{valeur:,.0f}".replace(",", " ") + " MAD"


if __name__ == "__main__":
    print("QUALITE", STAT_QUALITE)
    print("TYPES\n", df.dtypes)
    print("KPI\n", calculer_kpis(df))
```

# Annexe F - listing intégral du générateur CSV

Pour régénérer un jeu de données reproductible, utilisez `python generer_donnees.py`. Le fichier contient trois anomalies volontairement ajoutées pour la mission de qualité des données.

**FICHIER COMPLET - generer_donnees.py**

Code - partie 1/3 (coller les parties à la suite)

```
"""Genere un jeu de ventes fictives, avec 3 defauts volontaires."""
from datetime import date
from pathlib import Path
import random
import pandas as pd

RNG = random.Random(2026)
DOSSIER = Path(__file__).resolve().parent
CATALOGUE = {
    "Electronique": {
        "Casque Bluetooth": (320, 205),
        "Souris sans fil": (150, 75),
        "Clavier mecanique": (460, 280),
    },
    "Maison": {
        "Cafetiere": (720, 470),
        "Lampe LED": (185, 90),
        "Aspirateur": (1490, 990),
    },
    "Mode": {
        "Sac a dos": (420, 230),
        "Veste": (590, 335),
        "Baskets": (780, 455),
    },
    "Sport": {
        "Tapis de yoga": (190, 80),
        "Halteres": (350, 210),
        "Velo urbain": (2150, 1760),
    },
}
```

Code - partie 2/3 (coller les parties à la suite)

```
REGIONS = [
    "Casablanca-Settat", "Rabat-Sale-Kenitra", "Souss-Massa",
    "Marrakech-Safi", "Fes-Meknes", "Tanger-Tetouan-Al Hoceima",
]
POIDS_REGIONS = [31, 20, 13, 16, 12, 8]
JOURS = list(pd.date_range("2025-01-01", "2026-09-30", freq="D"))
POIDS_JOURS = [
    (1.18 if jour.year == 2026 else 1.0)
    * (1.75 if jour.month in (11, 12) else 1.0)
    * (1.10 if jour.weekday() >= 5 else 1.0)
    for jour in JOURS
]

lignes = []
ligne_id = 0
for numero in range(1, 2401):
    commande = f"CMD-{numero:05d}"
    jour = RNG.choices(JOURS, weights=POIDS_JOURS, k=1)[0]
    region = RNG.choices(REGIONS, weights=POIDS_REGIONS, k=1)[0]
    articles = RNG.randint(1, 3)
    for _ in range(articles):
        categorie = RNG.choice(list(CATALOGUE))
        produit = RNG.choice(list(CATALOGUE[categorie]))
        prix, cout = CATALOGUE[categorie][produit]
        quantite = RNG.randint(1, 4)
        remise = 0.10 if jour.month in (11, 12) and categorie == "Electronique" else 0.0
        ca = round(quantite * prix * (1 - remise), 2)
        ligne_id += 1
        lignes.append({
            "Ligne_ID": ligne_id,
```

Code - partie 3/3 (coller les parties à la suite)

```
            "Commande_ID": commande,
            "Date": jour.strftime("%Y-%m-%d"),
            "Produit": produit,
            "Categorie": categorie,
            "Region": region,
            "Quantite": quantite,
            "Prix_Unitaire": prix,
```

```
            "Cout_Unitaire": cout,
            "Remise_Pct": remise,
            "Chiffre_Affaires": ca,
        })

brut = pd.DataFrame(lignes)
# 3 anomalies intentionnelles pour la mission de qualite :
brut = pd.concat([brut, brut.iloc[[0]].copy()], ignore_index=True)
brut.loc[1, "Region"] = None
brut.loc[2, "Date"] = "date-invalide"
chemin = DOSSIER / "sales_data_brut.csv"
brut.to_csv(chemin, index=False, encoding="utf-8-sig")
print(f"CSV cree : {chemin.name}")
print(f"Lignes brutes : {len(brut)} ; commandes generees : 2400")
print("Indices : un doublon, une region manquante, une date invalide.")
```

# Annexe G - évaluation des acquis (sur 20)

| Compétence observable | Barème |
|---|---|
| Environnement VS Code, venv et commandes de base | 2 |
| Layout, composants et identifiants | 3 |
| Inputs, Outputs et callbacks réactifs | 4 |
| Nettoyage et calculs métier exacts | 4 |
| Graphiques pertinents et cohérents | 3 |
| Filtrage croisé et robustesse | 2 |
| Explication métier et documentation | 2 |
| **Total** | **20** |

## Trois défis bonus si vous terminez en avance

1. **Mission détective :** le dirigeant affirme que le CA total est le nombre de lignes multiplié par le prix moyen. Montrez pourquoi c’est faux sur cet historique de transactions.
2. **Mission designer :** changez la couleur du bandeau et l’arrondi des KPI dans `assets/style.css` ; expliquez quelle règle CSS a été modifiée.
3. **Mission data analyst :** ajoutez un cinquième KPI « taux de marque (%) » en utilisant la clé `taux_marque` déjà renvoyée par `calculer_kpis`. Précisez qu’il ne s’agit pas du taux de marge sur coût d’achat.

La réussite du TP ne se mesure pas au nombre de lignes copiées, mais à la capacité de prévoir le comportement du callback, expliquer un agrégat et identifier un résultat métier trompeur.